

CHAPTER 13

13 - 인증 + 역할 (JWT · Guard · RBAC)

Node.js · Express · NestJS 강의 · 2026

12장의 User는 이름만 있는 데이터였다. 상품의 판매자도 요청 본문으로 아무 id나 넣을 수 있었다. 이 장은 거기에 **진짜로** 그인을 더한다. 회원가입 때 비밀번호를 bcrypt로 해시하고, 로그인하면 JWT를 발급하고, 보호 라우트는 Guard로 막는다. 한 단계 더 나아가 ****역할(BUYER·SELLER·ADMIN)****로 누가 무엇을 할 수 있는지를 가른다.

가장 중요한 변화: 상품의 `sellerId` 가 더 이상 본문에 없다. ****로그인한 사용자(토큰)****가 판매자가 된다. 본문으로 받으면 남의 이름으로 등록할 수 있기 때문이다(위조).

이 장에서 달라진 것 (← 12장 대비)

구분	파일 / 내용
신규	<code>src/auth/</code> (회원가입·로그인·JWT·전략· <code>JwtAuthGuard</code>) · <code>src/common/</code> (<code>@CurrentUser</code> 데코레이터) · <code>prisma/seed.ts</code> (관리자 시드)
수정	<code>prisma/schema.prisma</code> — <code>User.password·role(enum)</code> · <code>products</code> — <code>sellerId</code> 를 본문에서 빼고 토큰에서, 등록·수정·삭제에 가드+소유권 · <code>categories</code> 생성은 ADMIN 전용 · <code>main.ts</code> 에 Swagger
의존성	<code>+@nestjs/jwt</code> <code>+@nestjs/passport</code> <code>+passport</code> <code>+passport-jwt</code> <code>+bcrypt</code> <code>+@nestjs/swagger</code>
엔드포인트	<code>+POST /auth/register</code> <code>+POST /auth/login</code> <code>+GET /auth/profile</code> · <code>GET /users/me</code> · 12장의 임시 <code>POST/GET /users</code> (목록·단건 조회)는 회원가입과 <code>/users/me</code> 로 대체(인증이 생겼으니 사용자를 아무나 직접 만들거나 전체 목록을 보는 라우트는 닫는다) · 상품/분류 라우트에 401·403 적용

0. 사전 준비 — PostgreSQL

PostgreSQL이 떠 있어야 한다(11장에서 접속한 그 서버). DBeaver에서 연결이 열리면 준비된 것이다.

1. 패키지 설치

```
npm i @nestjs/jwt@11 @nestjs/passport@11 passport@0.7 passport-jwt@4 bcrypt@6
@nestjs/swagger@11
npm i -D @types/passport-jwt@4 @types/bcrypt@6
```

메이저 버전은 NestJS 11 프로젝트와 짝이 맞는 것들이다(jwt·passport·swagger는 Nest 본체와 메이저를 맞춘다).

2. .env — JWT 시크릿 추가

.env 에 한 줄을 더한다(DATABASE_URL · PORT 는 11장 그대로).

```
# JWT 서명 시크릿 (운영에서는 길고 무작위한 값으로)
JWT_SECRET="dev-secret-change-me"
```

3. 모듈 생성

```
nest g module auth
nest g service auth --no-spec
nest g controller auth --no-spec
nest g guard auth/guards/jwt-auth --no-spec
# JwtStrategy, @CurrentUser 데코레이터는 직접 추가한다.
```

4. 스키마 — password · role + 마이그레이션

User 에 두 필드를 더하고, 권한 enum을 정의한다.

```
// 권한의 세 갈래. 게시판의 "작성자 본인"보다 한 단계 위 개념이다.
enum Role {
  BUYER // 구매자(기본)
  SELLER // 판매자 - 상품 등록
  ADMIN // 관리자 - 분류 관리
}

model User {
  id Int @id @default(autoincrement())
  email String @unique
  password String // bcrypt 해시 (13장에서 추가)
  name String
  role Role @default(BUYER) // 13장에서 추가
  products Product[]
  createdAt DateTime @default(now())
}
```

```
npm run prisma migrate dev --name add-auth
```

12장에서 만들어 본 User가 DB에 남아 있으면, 필수 컬럼 password 를 더할 수 없어 Prisma가 리셋을 물어볼 수 있다. 개발 DB이므로 동의하면 된다.

5. 시드 — 관리자 계정 + 기본 분류

`prisma/seed.ts` 를 만들고(코드는 해설서 13장), `package.json` 의 `scripts` 에 한 줄을 더한다.

```
"scripts": {  
  "seed": "ts-node prisma/seed.ts"  
}
```

```
npm run seed      # 관리자(admin@example.com / admin123) + 기본 분류
```

회원가입으로는 관리자를 만들 수 없다(권한 상승 방지). 관리자는 이 시드로만 생긴다. 시드는 upsert라 여러 번 돌려도 안전하다.

6. 코드 작성 순서

순 서	파일	내용
1	src/auth/dto/register.dto.ts · login.dto.ts	본문 검증 — role 은 BUYER·SELLER만 허용(ADMIN 금지)
2	src/auth/auth.service.ts	bcrypt 해시/검증 + JWT 발급(payload에 id · email · role)
3	src/auth/strategies/jwt.strategy.ts	Bearer 토큰 검증 → req.user = { id, email, role }
4	src/auth/guards/jwt-auth.guard.ts	AuthGuard("jwt") 상속
5	src/auth/auth.controller.ts · auth.module.ts	register·login·profile · JwtModule 등록(시크릿은 .env)
6	src/common/current-user.decorator.ts	토큰의 { id, email, role } (또는 한 필드)을 핸들 러에 주입
7	src/users/*	12장의 임시 POST/GET /users 제거 → GET /users/me 만
8	src/products/* · src/categories/*	등록·수정·삭제에 가드 + 역할 if + 소유권 검사 · sellerId 는 @CurrentUser("id") 에서
9	src/main.ts	Swagger 등록 → http://localhost:3000/docs

역할(SELLER·ADMIN) 검사는 컨트롤러의 **간단한 if **로 막는다. 이 if 들을 @Roles + RolesGuard 로 묶는 리팩터링은 17장에서 다룬다.

각 파일의 코드와 설명은 해설서 13장에 있다.

7. 실행 + 동작 테스트 (curl)

```
npm run start:dev      # → http://localhost:3000 (Swagger: /docs)
```

```

BASE=http://localhost:3000
login() { curl -s -X POST $BASE/auth/login -H "Content-Type: application/json" \
  -d '{"email":"'${1}',"password":"'${2}'" | jq -r .access_token; }

# 판매자·구매자 가입
curl -s -X POST $BASE/auth/register -H "Content-Type: application/json" \
  -d '{"email":"s1@example.com","password":"secret123","name":"판매자1","role":"SELLER"}'
curl -s -X POST $BASE/auth/register -H "Content-Type: application/json" \
  -d '{"email":"b@example.com","password":"secret123","name":"구매자"}' # role 생략 →
BUYER

ADMIN=$(login admin@example.com admin123)
S1=$(login s1@example.com secret123)
B=$(login b@example.com secret123)

# 1) 토큰 없이 상품 등록 → 401
curl -i -X POST $BASE/products -H "Content-Type: application/json" \
  -d '{"name":"x","description":"x","price":1,"stock":1,"categoryIds":[1]}' #
401

# 2) 구매자가 상품 등록 → 403 (역할 부족)
curl -i -X POST $BASE/products -H "Authorization: Bearer $B" -H "Content-Type:
application/json" \
  -d '{"name":"x","description":"x","price":1,"stock":1,"categoryIds":[1]}' #
403

# 3) 판매자가 상품 등록 → 성공. 응답의 seller가 토큰의 사용자다.
curl -s -X POST $BASE/products -H "Authorization: Bearer $S1" -H "Content-Type:
application/json" \
  -d '{"name":"무선 이어폰","description":"노이즈 캔슬
링","price":89000,"stock":10,"categoryIds":[1]}'

# 4) 분류 생성 - 판매자는 403, 관리자는 성공
curl -i -X POST $BASE/categories -H "Authorization: Bearer $S1" -H "Content-Type:
application/json" -d '{"name":"음향기기"}' # 403
curl -s -X POST $BASE/categories -H "Authorization: Bearer $ADMIN" -H "Content-Type:
application/json" -d '{"name":"음향기기"}'

# 5) 내 정보
curl -s $BASE/auth/profile -H "Authorization: Bearer $S1"

```

핵심 개념

401 · 403의 차이 — 세 단계 방어

토큰 없음 → 401 Unauthorized (JwtAuthGuard가 막음)
 토큰 있음 + 역할 부족 → 403 Forbidden (컨트롤러의 역할 if가 막음)
 역할 OK + 남의 상품 → 403 Forbidden (서비스의 소유권 검사가 막음)

`@UseGuards(JwtAuthGuard)` 로 가드는 로그인 여부만 본다. 역할(SELLER·ADMIN인가)은 컨트롤러에서 `if (user.role !== ...)` 로, 소유권(본인 상품인가)은 서비스에서 한 번 더 본다. 로그인 → 역할 → 소유권의 단계 방어다. 이 `if` 검사를 `@Roles` 데코레이터 + `RolesGuard` 로 묶는 리팩터링은 17장에서 다룬다.

역할은 토큰에 실어 둔다

로그인할 때 JWT payload에 `role` 을 담는다. 그래서 보호 라우트마다 DB를 다시 보지 않고, 토큰만 풀어 권한을 판단한다(`JwtStrategy.validate` → `req.user = { id, email, role }`).

판매자는 토큰에서 — 위조 방지

`CreateProductDto` 에 `sellerId` 가 없다. 컨트롤러가 `@CurrentUser("id")` 로 토큰의 사용자를 꺼내 서비스에 넘긴다. 클라이언트가 정할 수 없으니 남의 이름으로 등록할 수 없다.

핵심 파일

파일	역할
<code>auth/auth.service.ts</code>	bcrypt 해시/검증 + JWT 발급(payload에 role)
<code>auth/strategies/jwt.strategy.ts</code>	Bearer 토큰 검증 → <code>req.user</code> 구성
<code>products · categories</code> 컨트롤러	로그인은 <code>JwtAuthGuard</code> , 역할은 간단한 <code>if</code> 로 차단
<code>common/current-user.decorator.ts</code>	토큰의 <code>{ id, email, role }</code> 주입
<code>prisma/seed.ts</code>	관리자·기본 분류 시드

따라잡기 — 완성본으로 실행 (보조 경로)

수업의 기본 경로는 위처럼 직접 만드는 것이다. 따라오다 놓쳤다면, 수업 저장소의 13/ 폴더(완성 코드)로 이 장의 끝 상태에서 다시 시작할 수 있다.

```
cd 13
npm install
cp .env.example .env          # 사용자·비밀번호를 본인 PostgreSQL 값으로 수정
npx prisma migrate dev --name init  # 이 폴더 전용 DB(nest_shop_13) - 자동 생성이 안 되면
DBBeaver로 먼저 만든다
npm run seed
npm run start:dev
```

다음 장

- **14 - 캐싱 + 스케줄링**: 자주 읽히는 상품 상세를 Redis에 캐시하고, `@Cron` 으로 주기 작업을 돌린다.